Lesson 2.8 **Duration: 40 minutes**

Data Organization

Learning Objectives

- Choose appropriate data structures for different types of problems
- Organize data efficiently using combinations of lists, dictionaries, and tuples
- Design data models that represent real-world entities and relationships

Engage (5-7 minutes)

A librarian does not throw books into random piles—books are organized by genre, author, and subject for easy retrieval. Similarly, programmers must organize data strategically. Choosing the right data structure affects performance, readability, and maintainability. Today we learn to think about data organization as a design problem, selecting the best structure for each situation.

Explore (10-12 minutes)

Consider storing a class register. List three different ways to organize it: as a list of lists, as a list of dictionaries, and as a dictionary of lists. Write out examples of each with five students. Discuss with a partner: which approach makes it easiest to find a student by name? Which makes it easiest to list all marks for a subject? Which is most memory-efficient?

Explain (10-12 minutes)

Data organization is about choosing the right structure for the right job. Lists are best for ordered collections accessed by position. Dictionaries excel at key-based lookups. Tuples protect data from modification. Sets handle uniqueness and mathematical operations. Nested structures combine these strengths: a list of dictionaries represents a collection of records, a dictionary of lists groups related data, and a dictionary of dictionaries models hierarchical data. Good data organization leads to cleaner code, better performance, and easier maintenance.

Elaborate (8-10 minutes)

Design data structures for: a library system (books, authors, borrowing records), a hospital management system (patients, doctors, appointments), and a weather monitoring system (locations, daily readings, alerts). For each, write the structure as Python code, explain why you chose each data type, and implement functions to add, search, and generate reports. Compare your designs with classmates and discuss trade-offs.

Evaluate (5-8 minutes)

Given the requirement “store a shopping cart where each item has a name, price, and quantity,” design two different data structures and implement an `add_to_cart` and `calculate_total` function for each. Which design is more efficient for calculating the total? Which is easier to display as a receipt? Justify your choices.